UNIVERSITY OF TECHNOLOGY EINDHOVEN

PROJECT 2

5LSH0 COMPUTER VISION AI AND 3D DATA ANALYSIS

3D Point Cloud Filtering

Names
Joost LELIVELD

Student numbers
1609726

June 18, 2026

Contents

1 Point Cloud Importing

1.1 Point cloud generation using a stereo camera

Since a single image does not have any depth information, stereo vision systems take left and right images of a scene from different viewpoints. These 2 images provide displacement of corresponding points in image pair (disparity). From this disparity the depth can be calculated. The 2 cameras have different optical centers and thus have different virtual image planes. The virtual image plane is a projection of the actual image through the respective optical center. The displacement of a point P with respect to the 2 image planes is the disparity. To go from disparity to depth you can use the formula z = b*f / p*d. with p = pixel width, b = distance between the optical centres, f = focal length and d = disparity. This gives a disparity/ depth map for areas and noise. The depth information is combined with 2D image data to create the 3D (x,y,z) point cloud.

1.2 Reasons for noise

In a stereo camera there are several causes for noise. Firstly, as mentioned in question 1, to get the disparity the left and right images are matched. Repetitive patterns or occlusions can lead to inaccurate matches and thus disparity values. This faulty disparity is then still used for depth calculation which will then create noisy depth values. This is especially difficult in areas with low texture (uniform surfaces such as a wall with 1 colour). When most of the image is uniform wall it becomes almost impossible to match points in the images, creating noise in the point cloud because of the again incorrect depth calculations. Other environmental factors could also cause noise. Reflective surfaces, shadows, motion artifacts and lens distortions all affect matching points together and thus the calculation of the disparity and depth.

1.3 Node that subscribes to point cloud topics

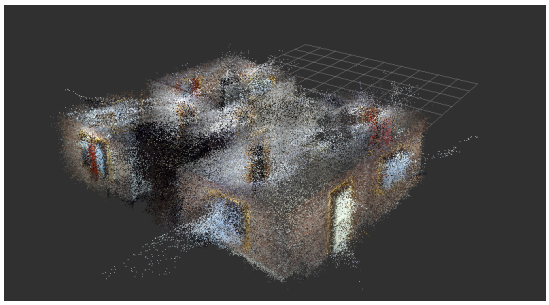
I used the already created workspace "Assignment_ws". There I created a new package "point_cloud_filter" using the steps from the practice exercises. In the listener.py file I made changes so it is able to subscribe to point cloud topics. The final file also does the filtering steps already. But the standard code I changed to subscribe to the point cloud topics was:

```
import rospy
from std_msgs.msg import String
from sensor_msgs.msg import PointCloud2
import sensor_msgs.point_cloud2 as pc2
def callback(msg):
    # Basic logging for simple implementation of question 3
    rospy.loginfo("Received a PointCloud2 message")
    rospy.loginfo("Header: %s", msg.header)
    rospy.loginfo("Height: %d, Width: %d", msg.height,msg.width)
    rospy.loginfo("Fields: %s", msg.fields)


def point_cloud_filtering_node():

    # In ROS, nodes are uniquely named. If two nodes with the same
    # name are launched, the previous one is kicked off. The
    # anonymous=True flag means that rospy will choose a unique
    # name for our 'listener' node so that multiple listeners can
    # run simultaneously.
    rospy.init_node('point_cloud_filtering_node', anonymous=True)
```

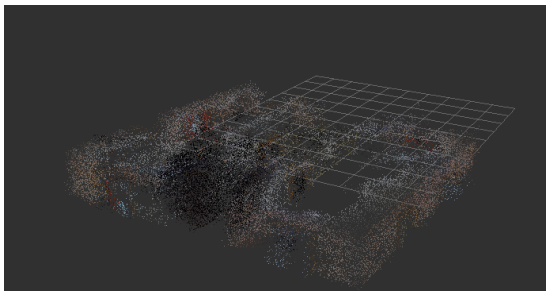
```
    rospy.Subscriber('/cloud_map', PointCloud2, callback)

    # spin() simply keeps python from exiting until this node is stopped
    rospy.spin()

if __name__ == '__main__':
    point_cloud_filtering_node()
```

2 Design your filtering node in ROS

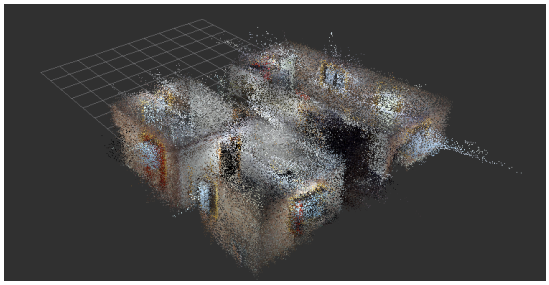
Link to my point cloud bag (only accessible with TU/e account)



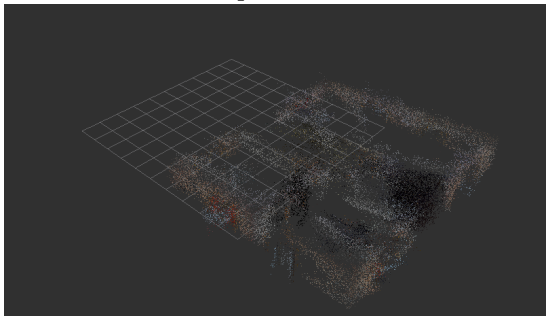
(a) Unfiltered point cloud view 1



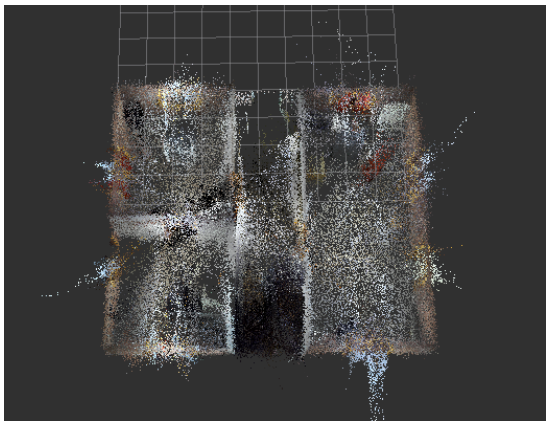
(b) Filtered point cloud view 1



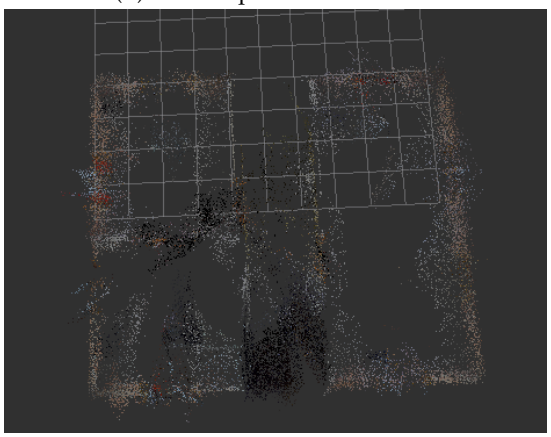
(c) Unfiltered point cloud view 2



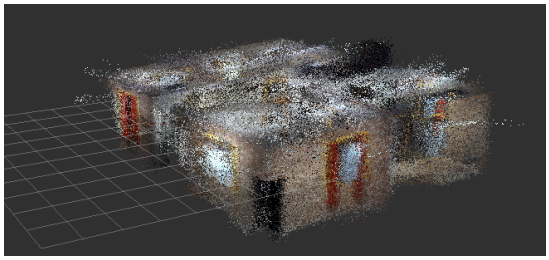
(d) Filtered point cloud view 2



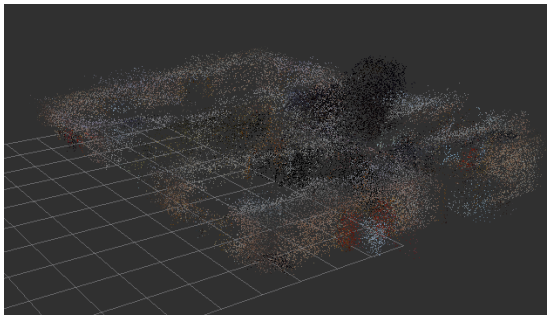
(e) Unfiltered point cloud view 3



(f) Filtered point cloud view 3



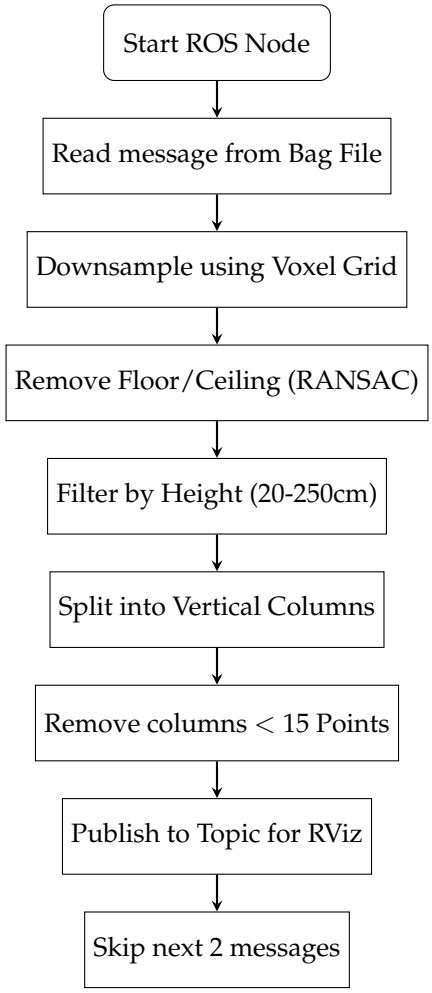
(g) Unfiltered point cloud view 4



(h) Filtered point cloud view 4

Figure 1: Side-by-side views of the unfiltered and the filtered point cloud

2.1 Filtering procedure



The filtering procedure that I ended up with is quite simple since it has to process all the points very quickly. The code can be found in the appendix. We were not interested in floors, ceilings and furniture, but in windows, walls and doors. These can be recognized by filtering based on vertical columns.

The node is started in the terminal after which I play the given bag file with the raw point cloud. When the node receives a message from the published topic containing the points for that time, all points will go through some procedures. Firstly all the points in that message are downsampled using a voxel grid. This reduces the number of points by grouping them and this allows for faster processing and already some noise reduction.

Then an attempt is made to remove the floor or ceiling. By assuming that the largest horizontal plane is the floor or ceiling we can remove that using RANSAC. For 20 iterations three points are sampled and a plane equation is computed and for that the plane the number of points belonging to that plane is computed. The plane with the largest number of points is removed. This algorithm is not ideal since 20 iterations might not be enough to find the largest plane and more iterations would cause the runtime to increase greatly.

After that a more basic attempt is done to make sure the floor and ceiling points are removed by filtering on the height. Points outside of the range 20-250cm are removed as they are not in-

teresting to us.

Lastly the point cloud is split into 100 vertical columns. Each column is analyzed and there is a threshold for 15 points, if there are less than 15 points in a vertical column it means that it is probably not a wall, door or window. By splitting the data in vertical columns and thresholding the number of points in that column you can filter out horizontal planes. Columns that only contain part of a ceiling, floor and maybe some furniture will not have a lot of points and will fall below that threshold.

Since these steps longer than it takes the bag file to publish new messages a queue is created. However this queue can become too large and it will cost too much memory or it is too slow in showing the new data. That is why I decided to process only one in every 5 messages. This prevents a queue from building up and thus you will see more recent results faster and you will be able to see the whole point cloud before the memory of the queue gets too large.

Finally this point cloud is published in a topic which is subscribed to in RVIZ for visualisation.

2.2 Further speed and performance improvements

Currently the size of the messages keep increasing since the points are accumulated instead of being sent incrementally. One way to improve the speed and performance is by focusing more on the recent points rather than on all the points. Since old points might be updated to a more accurate position I could not just look at the new points that were being sent in the message, however recomputing all the filters and outliers for every point is very expensive while changes to the first points might be minimal. For example in detecting the largest plane for removing the floor or the ceiling we could detect the floor or ceiling and then filter new points that fall within that plane instead of recomputing a single plane. However this does require the messages to contain enough information or the messages should be combined.

Increasing the size of the voxels will also speed up the filtering process. By combining more points all computations can be done more quickly. Another improvement would be to create the code in C++ since Python computations are a lot slower. Right now even with simple code the messages are too fast and a queue builds up.

To improve the performance the hyperparameters could be tweaked since these have not been thoroughly tested and could play a big role. This however does make the filtering less general for other point clouds. More complex methods such as DBSCAN, KDTREE or other clustering methods could also improve the performance.

3 Appendix

```
#!/usr/bin/env python
# -*- coding: utf-8 -*-
# Software License Agreement (BSD License)
#
# Copyright (c) 2008, Willow Garage, Inc.
# All rights reserved.
#
# Redistribution and use in source and binary forms, with or without
# modification, are permitted provided that the following conditions
# are met:
#
```

```
#  * Redistributions of source code must retain the above copyright
#    notice, this list of conditions and the following disclaimer.
#  * Redistributions in binary form must reproduce the above
#    copyright notice, this list of conditions and the following
#    disclaimer in the documentation and/or other materials provided
#    with the distribution.
#  * Neither the name of Willow Garage, Inc. nor the names of its
#    contributors may be used to endorse or promote products derived
#    from this software without specific prior written permission.
#
# THIS SOFTWARE IS PROVIDED BY THE COPYRIGHT HOLDERS AND CONTRIBUTORS
# "AS IS" AND ANY EXPRESS OR IMPLIED WARRANTIES, INCLUDING, BUT NOT
# LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS
# FOR A PARTICULAR PURPOSE ARE DISCLAIMED. IN NO EVENT SHALL THE
# COPYRIGHT OWNER OR CONTRIBUTORS BE LIABLE FOR ANY DIRECT, INDIRECT,
# INCIDENTAL, SPECIAL, EXEMPLARY, OR CONSEQUENTIAL DAMAGES (INCLUDING,
# BUT NOT LIMITED TO, PROCUREMENT OF SUBSTITUTE GOODS OR SERVICES;
# LOSS OF USE, DATA, OR PROFITS; OR BUSINESS INTERRUPTION) HOWEVER
# CAUSED AND ON ANY THEORY OF LIABILITY, WHETHER IN CONTRACT, STRICT
# LIABILITY, OR TORT (INCLUDING NEGLIGENCE OR OTHERWISE) ARISING IN
# ANY WAY OUT OF THE USE OF THIS SOFTWARE, EVEN IF ADVISED OF THE
# POSSIBILITY OF SUCH DAMAGE.
#
# Revision $Id$

import rospy
import numpy as np
from sensor_msgs.msg import PointCloud2, PointField
import sensor_msgs.point_cloud2 as pc2
from std_msgs.msg import Header
import random

#These are hyperparameters that I played around with to get better results
pub = None
voxel_dict = {}
voxel_size = 0.05 # the larger the more downsampling

SKIP = 5 #if this was 1 then it would not get to the end
msg_counter = 0 #to check how far we get
MAX_PLANE_DIST = 0.02
MAX_ITER = 20
Z_MIN, Z_MAX = 0.2, 2.5 #min and max height
NUM_BINS = 100
MIN_NR_OF_POINTS = 10


def voxel_grid(points): #merge points using a voxel grid
    global voxel_dict, voxel_size
    if voxel_size <= 0:#no voxels just use the normal points
        for i in range(points.shape[0]):
            key = ("not_voxel", i)
            voxel_dict[key] = points[i]
        return
    coordinates = points[:, :3]
```

```
    voxel_index = np.floor(coordinates /voxel_size).astype( np.int32)
    for i, idx in enumerate(voxel_index):
        key = tuple(idx)
        voxel_dict[key] = points[i]
    return np.array(list(voxel_dict.values()), dtype=np.float32)


def remove_largest_plane(points, threshold=MAX_PLANE_DIST, max_iter=MAX_ITER):
    if points.shape[0] < 4:
        return points
    coords = points[:, :3]
    nr_of_points = coords.shape[0]

    nr_in_largest_plane = 0
    best_plane_filter = None

    for _ in range(max_iter):

        idx = np.random.choice(nr_of_points, 3, replace=False)
        point_1, point_2, point_3 = coords[idx]

        normal_plane = np.cross(point_2 - point_1, point_3 - point_1)
        normal_plane /= np.linalg.norm(normal_plane)

        bias = -np.dot(normal_plane, point_1)
        distances = np.abs(np.dot( coords , normal_plane) + bias)

        in_plane = distances < threshold
        count_in = np.sum(in_plane)

        if count_in > nr_in_largest_plane:
            nr_in_largest_plane = count_in
            best_plane_filter = in_plane

    if best_plane_filter is None:
        return points

    return points[~best_plane_filter]


def z_filter(points, z_min=Z_MIN, z_max=Z_MAX):
    if points.size == 0:
        return points
    z = points[:,2]
    height_filter = (z >z_min) & (z < z_max)
    return points[height_filter]


def filter_columns(points, num_bins=NUM_BINS, threshold=MIN_NR_OF_POINTS):
    if points.size == 0:
        return points

    coords = points[:, :3]
    min_x, max_x = coords[:, 0].min() , coords[:, 0].max()
```

```
    min_y, max_y = coords[:, 1].min() , coords[:, 1].max()

    column_edges_x = np.linspace(min_x, max_x, num_bins)
    column_edges_y = np.linspace(min_y, max_y, num_bins)
    i_x_assignment = np.clip(np.digitize(coords[:,0], column_edges_x)-1, 0, num_bins-1)
    i_y_assignment = np.clip(np.digitize(coords[:,1], column_edges_y)-1, 0, num_bins-1)

    bin_index = i_x_assignment * num_bins + i_y_assignment
    counts = np.bincount(bin_index, minlength=num_bins*num_bins)
    nr_in_column = counts[bin_index]

    mask = nr_in_column >= threshold
    return points[mask]


def callback(msg):

    global msg_counter

    # Skip messages to reduce backlog
    msg_counter += 1
    if msg_counter % SKIP != 0:
        rospy.loginfo("Skipped message %d ", msg_counter)
        return

    new_pts = np.array([p for p in pc2.read_points(msg, skip_nans=True)], dtype=np.float
    grid = voxel_grid(new_pts)

    no_floor = remove_largest_plane(grid, threshold=MAX_PLANE_DIST, max_iter=MAX_ITER)

    vertical = z_filter(no_floor, z_min=Z_MIN, z_max=Z_MAX)

    final_cloud = filter_columns(vertical, num_bins=NUM_BINS, threshold=MIN_NR_OF_POINTS

    publish_cloud(final_cloud)

def publish_cloud(points):
    global pub
    header = Header()

    header.stamp = rospy.Time.now()
    header.frame_id = "map"

    fields = [
        PointField("x",   0,  PointField.FLOAT32, 1),
        PointField("y",   4,  PointField.FLOAT32, 1),
        PointField("z",   8,  PointField.FLOAT32, 1),
        PointField("rgb", 12, PointField.FLOAT32, 1),
    ]

    point_cloud = points.tolist()
    msg = pc2.create_cloud(header, fields, point_cloud)
    pub.publish(msg)
```

```
def point_cloud_filtering_node():
    global pub
    rospy.init_node('point_cloud_filtering_node', anonymous=True)

    pub = rospy.Publisher("/filtered_cloud", PointCloud2, queue_size=10)
    rospy.Subscriber("/cloud_map", PointCloud2, callback, queue_size=50)

    rospy.loginfo("Node for filtering point clouds started.")
    rospy.spin()

if __name__ == '__main__':
    point_cloud_filtering_node()
```